Temporal Difference Methods in Reinforcement Learning

RL Assignment Report — S&B Chapters 6 & 7

---

1 TD Prediction — TD(0)

1.1 Motivation and Core Idea

In reinforcement learning, the goal of *prediction* is to estimate the state-value function $V^\pi(s)$, the expected cumulative discounted reward when following policy π from state s. Two classical approaches exist:

- **Monte Carlo (MC)**: Wait until the end of an episode, collect the full return $G_t = R_{t+1} + \gamma R_{t+2} + \cdots$, and update $V(S_t)$ towards it.
- **Dynamic Programming (DP)**: Update $V(S_t)$ using the known model of the environment.

TD(0) combines the strengths of both. Like MC it learns directly from experience without a model; like DP it *bootstraps* — it updates estimates using other estimates rather than waiting for a full return.

1.2 The TD(0) Update Rule

After taking action A_t in state S_t, observing reward R_{t+1} and landing in S_{t+1}, TD(0) immediately updates:

$$V(S_t) \leftarrow V(S_t) + \alpha \underbrace{\left[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\right]}_{\delta_t \text{ (TD error)}} \tag{1}$$

The term $R_{t+1} + \gamma V(S_{t+1})$ is called the **TD target**. It is a biased but low-variance estimate of the true return: biased because $V(S_{t+1})$ is itself an approximation, but low-variance because it uses only one step of actual experience.

1.3 The TD Error as a Prediction-Error Signal

The quantity

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

is the **TD error**. It measures the discrepancy between what the agent *expected* to get (its current value estimate $V(S_t)$) and what it *actually experienced* in the next step ($R_{t+1} + \gamma V(S_{t+1})$).

Intuitively, δ_t is a surprise signal: a large positive error means the outcome was better than predicted, prompting an upward revision; a large negative error means it was worse. This mirrors the *reward prediction error* signal found in dopaminergic neurons in neuroscience, giving TD learning biological plausibility beyond its algorithmic motivation.

1.4 Comparison with Monte Carlo Prediction

Property	TD(0)	Monte Carlo
Update timing	Online (every step)	End of episode
Target	$R_{t+1} + \gamma V(S_{t+1})$	Full return G_t
Bootstrapping	Yes	No
Bias	Higher (uses estimates)	None (uses true returns)
Variance	Lower	Higher
Works with continuing tasks	Yes	No

The key trade-off is **bias vs. variance**. MC has zero bias (it uses the true return) but high variance because the return depends on many random steps. TD introduces bias by substituting $V(S_{t+1})$ for the tail of the return, but this dramatically reduces variance since it depends on only one transition.

1.5 Convergence

Under a tabular representation and step sizes satisfying the Robbins–Monro conditions ($\sum_t \alpha_t = \infty$, $\sum_t \alpha_t^2 < \infty$), TD(0) converges with probability 1 to V^π for any fixed policy π (S&B Theorem 6.1). With a constant step size α, it converges in the mean to a neighbourhood of V^π, which is useful in non-stationary environments.

2 SARSA — On-Policy TD Control

2.1 From Prediction to Control

Prediction alone is not enough for an agent to act optimally. *Control* requires estimating the **action-value function** $Q^\pi(s, a)$, the expected return after taking action a in state s and thereafter following π. Once Q is well-estimated, a greedy policy $\pi'(s) = \arg\max_a Q(s, a)$ is immediately available without needing a model of the environment.

SARSA is the natural extension of TD(0) to action values. The name reflects the five quantities used per update: $(S_t,\ A_t,\ R_{t+1},\ S_{t+1},\ A_{t+1})$.

2.2 The SARSA Update

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big[R_{t+1} + \gamma\, Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\big] \tag{2}$$

After choosing A_t from S_t (using the current policy, e.g. ε-greedy), observing R_{t+1} and S_{t+1}, the agent *selects* its next action A_{t+1} from the same policy before performing the update. This is the crucial distinction from Q-Learning: the next action is sampled from the *behaviour policy*, making SARSA **on-policy**.

The full tuple $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$ is required because the TD target for action values, $R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$, depends on what action will actually be taken next. Without A_{t+1} being drawn from π, the update would not correctly evaluate the on-policy value.

2.3 Policy Improvement with ε-Greedy Exploration

SARSA operates within an on-policy framework: the same policy is used to generate data and to be improved. To ensure all state-action pairs continue to be visited, an ε-**greedy** policy is used:

$$\pi(a \mid s) = \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}| & \text{if } a = \arg\max_{a'} Q(s, a') \\ \varepsilon/|\mathcal{A}| & \text{otherwise} \end{cases}$$

With probability $1-\varepsilon$ the agent *exploits* (acts greedily) and with probability ε it *explores* (acts randomly). After each update the policy is implicitly improved by the new Q estimates.

2.4 Convergence

SARSA converges to the optimal policy π^* and optimal action-value function Q^* provided the policy satisfies **GLIE** (Greedy in the Limit with Infinite Exploration): every state-action pair is visited infinitely often, and the policy becomes greedy in the limit. A common realisation is $\varepsilon_t \to 0$ sufficiently slowly (e.g. $\varepsilon_t = 1/t$).

2.5 Backup Diagram

The SARSA backup diagram involves a root state-action node (S_t, A_t), a transition to an environment node yielding reward R_{t+1}, then a next state S_{t+1}, and finally the next action A_{t+1} sampled from the policy. The update flows back from (S_{t+1}, A_{t+1}) to (S_t, A_t), capturing one full *experience tuple*.

3 Q-Learning — Off-Policy TD Control

3.1 The Off-Policy Idea

SARSA evaluates the policy *actually followed* by the agent. Q-Learning breaks this coupling: the agent may explore freely (using any behaviour policy b), yet it learns about the *optimal* policy $\pi^* = \arg\max Q^*$ directly. This is the defining property of **off-policy** learning.

3.2 The Q-Learning Update

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\Big] \tag{3}$$

The key difference from SARSA is the $\max_a$ operator in the TD target. Instead of using $Q(S_{t+1}, A_{t+1})$ where A_{t+1} is drawn from the behaviour policy, Q-Learning always bootstraps from the *best possible* next action. This means the target is computing the value of the **greedy (target) policy**, regardless of what the agent actually does next.

3.3 Why the max Makes It Off-Policy

In SARSA, $A_{t+1} \sim \pi(\cdot \mid S_{t+1})$, so the update evaluates π. In Q-Learning, $\max_a Q(S_{t+1}, a)$ implicitly evaluates the greedy policy over Q — a *different* policy from the ε-greedy behaviour policy used to collect data. Because the target policy (greedy) and behaviour policy (ε-greedy) are decoupled, Q-Learning is off-policy and converges to Q^* directly, without any GLIE requirement on the behaviour policy beyond sufficient exploration.

3.4 Cliff Walking: SARSA vs. Q-Learning

The Cliff Walking gridworld (S&B Example 6.6) illustrates the practical consequence of this distinction. The agent must travel from a start cell to a goal cell along the edge of a cliff. Falling off the cliff gives reward -100; every other step gives -1.

- **Q-Learning** learns the *optimal* path: a direct route along the cliff edge. Because it evaluates the greedy policy (which never takes the random exploratory steps that might fall off), it converges to the shortest, highest-reward trajectory.
- **SARSA** learns a *safer* path: it routes further from the cliff. Because SARSA evaluates the actual ε-greedy policy being followed, it accounts for the non-zero chance of accidentally

stepping off the cliff due to exploration. The resulting policy avoids this risk at the cost of a slightly longer route.

Neither solution is universally better. Q-Learning finds the theoretically optimal policy, but during training its cumulative reward is lower because it walks close to the cliff while still exploring. SARSA achieves better online performance during learning because its policy is adapted to the exploratory behaviour.

3.5 Backup Diagram

The Q-Learning backup diagram is similar to SARSA's up to the next state. However, instead of following a single sampled action from S_{t+1}, it shows *all* actions from S_{t+1} fanning out, with the maximum value selected. This reflects the greedy maximisation rather than sampling from π.

4 n-Step TD Methods

4.1 Bridging TD(0) and Monte Carlo

TD(0) uses a one-step return as its target. MC uses the full episode return. Neither is always optimal: one step may be too myopic; a full episode may have too much variance. **n-step TD** methods generalise both by looking n steps ahead before bootstrapping.

4.2 The n-Step Return

The n-step return starting from time t is:

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}) \tag{4}$$

It accumulates n actual rewards, then bootstraps from the value estimate at S_{t+n}. The n-step TD update is then:

$$V(S_t) \leftarrow V(S_t) + \alpha\big[G_t^{(n)} - V(S_t)\big]$$

4.3 Recovering TD(0) and Monte Carlo

- $n = 1$: $G_t^{(1)} = R_{t+1} + \gamma V(S_{t+1})$, which is exactly the TD(0) target.
- $n \to \infty$: The bootstrap term $\gamma^n V(S_{t+n})$ vanishes (assuming $\gamma < 1$), and the target reduces to the full MC return G_t.

Thus n is a continuous dial between TD(0) and MC.

4.4 Bias–Variance Trade-off

Increasing n reduces bias: more actual rewards are used and less reliance is placed on potentially inaccurate value estimates. However, it increases variance: more random transitions contribute to the target. The optimal n is problem-dependent and is often treated as a hyperparameter. In practice, intermediate values (e.g. $n \in \{4, 8, 16\}$) frequently outperform both extremes.

4.5 n-Step SARSA

The n-step idea extends naturally to action values for control. The n-step SARSA return is:

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n Q(S_{t+n}, A_{t+n})$$

and the update replaces the bootstrapped state value with the action-value estimate at the end of the n-step trajectory. The agent must store the trajectory $(S_t, A_t, R_{t+1}, \ldots, S_{t+n}, A_{t+n})$ before updating, introducing a delay of n steps relative to TD(0).

5 TD(λ) and Eligibility Traces

5.1 Unifying All n-Step Returns

n-step methods require choosing a single value of n. TD(λ) takes a more principled approach: rather than picking one n, it *averages over all* n-step returns, weighted geometrically by λ^{n-1}.

5.2 The λ-Return

$$G_t^\lambda = (1-\lambda)\sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)} \tag{5}$$

The factor $(1-\lambda)$ normalises the weights so they sum to 1. The parameter $\lambda \in [0,1]$ controls how much weight is placed on longer-horizon returns:

- $\lambda = 0$: All weight falls on the 1-step return $G_t^{(1)}$, recovering TD(0).
- $\lambda = 1$: The weights reproduce the full MC return (under offline/batch updates at the end of an episode).
- **Intermediate** λ: Smoothly interpolates between the two, providing flexible bias–variance control.

5.3 Eligibility Traces: An Efficient Implementation

Computing G_t^λ naïvely requires storing and summing all future n-step returns — computationally prohibitive online. **Eligibility traces** solve this by maintaining a *backward view*: a memory $e_t(s)$ for each state that tracks how recently and frequently s has been visited.

The TD error at each step is:

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

The eligibility trace for each state s is updated first:

$$e_t(s) = \gamma\lambda\, e_{t-1}(s) + \mathbf{1}[S_t = s]$$

Then *every* state value is updated in proportion to its trace:

$$V(s) \;\leftarrow\; V(s) + \alpha\,\delta_t\, e_t(s) \qquad \forall s$$

The trace $e_t(s)$ decays by $\gamma\lambda$ each step, so recently and frequently visited states receive larger updates. This elegantly propagates credit backwards through time without storing the full trajectory.

5.4 Accumulating vs. Replacing Traces

Two common variants exist for how the trace is incremented when a state is revisited:

- **Accumulating traces**: Add 1 on each visit: $e_t(s) \leftarrow \gamma\lambda\, e_{t-1}(s) + 1$ if $S_t = s$.
  Traces can grow large for frequently visited states.
- **Replacing traces**: Cap at 1 on each visit: $e_t(s) \leftarrow 1$ if $S_t = s$.
  This prevents runaway growth and often performs better in practice.

5.5 Computational Advantage

The key practical benefit of eligibility traces is efficiency. An explicit n-step method requires: (i) storing the last n transitions in memory, (ii) waiting n steps before updating, and (iii) performing a separate rollout for each n. Eligibility traces achieve the effect of a weighted average over all n-step lookbacks with a *single forward pass*, updating every state at every step using only the current TD error and the cached trace values. The computational cost per step is $O(|\mathcal{S}|)$, independent of the effective lookahead horizon encoded by λ — a significant advantage over explicit n-step rollouts, particularly for large λ (long effective horizons).

5 Conclusion

The TD family of algorithms forms a principled progression. TD(0) establishes the core idea of bootstrapped online learning from experience. SARSA and Q-Learning extend it to control, differing in whether they evaluate the current exploratory policy (on-policy) or the optimal greedy policy (off-policy). n-step TD generalises the lookahead depth, revealing TD and MC as extremes on a continuous spectrum. TD(λ) and eligibility traces unify this spectrum elegantly and efficiently, distributing credit through time with minimal computational overhead. Together they constitute the foundational toolkit of model-free reinforcement learning.